

GUÍA DE TRABAJO — ANÁLISIS DE TESTING EN BIBLIOTECA Y TRANSICIÓN A MICROSERVICIOS

DSY1103 — Desarrollo FullStack I

1. Contexto de la actividad

Durante esta etapa trabajaremos con el proyecto **Biblioteca Salas Duoc**, el cual ya cuenta con ejemplos de pruebas unitarias implementadas para analizar.

El objetivo principal **no es construir desde cero todos los test**, ya que existen ejemplos previamente desarrollados. La finalidad es que los estudiantes comprendan cómo fueron diseñadas las pruebas, qué capa se está probando, qué dependencias se simulan y qué resultados se esperan.

Posteriormente, este mismo análisis será aplicado a proyectos basados en microservicios. Para ello, el docente está preparando un proyecto llamado **DoggySpa**, el cual será entregado más adelante como ejemplo inicial para trabajar pruebas unitarias en microservicios reales.

2. Material disponible

Los estudiantes trabajarán con los siguientes recursos:

Material de análisis en Biblioteca

TESTING - Analizar Services Test - biblioteca.salas.duoc

Este material contiene ejemplos de pruebas enfocadas en la capa **Service** del proyecto Biblioteca.

TESTING - CONTROLLER - biblioteca.salas.duoc

Este material contiene ejemplos de pruebas enfocadas en la capa **Controller**, utilizando pruebas sobre endpoints REST.

Guías de apoyo

3.1.6 Guía práctica - Proyecto Biblioteca Duoc Parte 1 - DataFaker

Esta guía explica cómo poblar el proyecto Biblioteca con datos falsos mediante DataFaker. El documento plantea que el proyecto ya posee models, repositories, services y controllers, y que se incorporan datos falsos para disponer de información inicial de prueba.

3.1.7 Guía práctica - Proyecto Biblioteca Duoc Parte 2 - Análisis a los test

Esta guía orienta el análisis de pruebas unitarias con JUnit 5 y Mockito, explicando la separación de ambientes de desarrollo y prueba para evitar ejecutar test sobre la misma base de datos de trabajo.

3.1.5 Pruebas unitarias en proyectos de microservicios

Este material entrega la base conceptual de testing, explicando la importancia de las pruebas unitarias, el riesgo de no tener testing y el valor de probar unidades pequeñas de código de forma aislada.

3. Objetivo general

Analizar pruebas unitarias ya implementadas en el proyecto Biblioteca y comprender cómo trasladar ese mismo razonamiento a un contexto de microservicios.

4. Objetivos específicos

Al finalizar la actividad, el estudiante deberá ser capaz de:

1. Identificar la capa que se está probando.
 2. Diferenciar pruebas de Service y pruebas de Controller.
 3. Reconocer el uso de mocks.
 4. Explicar el propósito de `when()`, `thenReturn()`, `assertEquals()`, `assertNotNull()` y `verify()`.
 5. Comprender el uso de `MockMvc` en pruebas de Controller.
 6. Identificar errores comunes al ejecutar pruebas.
 7. Relacionar el ejemplo Biblioteca con una futura implementación en microservicios.
 8. Prepararse para aplicar este patrón en el proyecto DoggySpa.
-

5. Por qué no se crearán todos los test desde cero

En esta actividad no se espera que los estudiantes desarrollen nuevamente todos los test del proyecto Biblioteca.

La razón es que el material ya contiene ejemplos contruidos.

Por lo tanto, rehacerlos completamente podría hacer que se pierda tiempo en copiar código, en lugar de comprender su funcionamiento.

El foco será:

Leer

Analizar

Explicar

Comprender

Adaptar

No se busca solamente que el test funcione, sino que el estudiante pueda explicar qué está ocurriendo en cada parte.

6. Recordatorio conceptual: ¿Qué es una prueba unitaria?

Una prueba unitaria es una prueba automatizada que verifica el comportamiento de una unidad pequeña del código, normalmente un método o una clase.

En proyectos Spring Boot, una unidad puede ser:

- Un método del Service
- Un método del Controller
- Una regla de negocio
- Una validación
- Una respuesta esperada de un endpoint

El material de microservicios explica que, sin pruebas unitarias, pueden aparecer errores no detectados, regresiones, mayor dificultad para refactorizar y aumento del costo de mantenimiento.

7. Análisis de pruebas en capa Service

7.1 Qué es la capa Service

La capa Service contiene la lógica de negocio de la aplicación.

En un proyecto típico Spring Boot, la estructura es:

Controller -> Service -> Repository

El Controller recibe la petición HTTP.

El Service procesa la lógica.

El Repository accede a la base de datos.

7.2 Qué se analiza en los Service Test

En los archivos de Service Test del proyecto Biblioteca se deben revisar pruebas asociadas a servicios como:

CarreraService

SalaService

TipoSalaService

ReservaService

Cada prueba busca validar que el Service llame correctamente al Repository y devuelva el resultado esperado.

7.3 Ejemplo conceptual: findAll()

Una prueba de findAll() normalmente busca validar que el servicio devuelva una lista de registros.

Ejemplo conceptual:

```
when(carreraRepository.findAll())  
    .thenReturn(List.of(new Carrera("1", "Ingeniería")));
```

```
List<Carrera> carreras = carreraService.findAll();
```

```
assertNotNull(carreras);
```

```
assertEquals(1, carreras.size());
```

7.4 Explicación del ejemplo

La línea:

```
when(carreraRepository.findAll())  
    .thenReturn(List.of(new Carrera("1", "Ingeniería")));
```

indica que el Repository simulado debe devolver una lista con una carrera.

La línea:

```
List<Carrera> carreras = carreraService.findAll();
```

ejecuta el método real del Service.

La línea:

```
assertNotNull(carreras);
```

verifica que el resultado no sea nulo.

La línea:

```
assertEquals(1, carreras.size());
```

verifica que la lista tenga un elemento.

7.5 Qué se debe observar en cada Service Test

Para cada prueba de Service, el estudiante debe responder:

Pregunta	Respuesta esperada
¿Qué clase se está probando?	Servicio correspondiente
¿Qué método se está probando?	findAll, findById, save, delete, etc.
¿Qué dependencia se simula?	Repository
¿Qué datos se preparan?	Objetos simulados
¿Qué método se ejecuta?	Método del Service
¿Qué assert se utiliza?	assertNotNull, assertEquals, assertTrue, etc.
¿Qué resultado se espera?	Lista, objeto, eliminación, excepción o valor esperado
¿La prueba usa la base de datos real?	No debería
¿Qué podría fallar?	Mock mal configurado, método incorrecto, dato esperado distinto

8. Uso de Mockito en Service Test

8.1 Qué es un mock

Un mock es un objeto simulado.

Permite probar una clase sin depender de una base de datos, de otro servicio o de una conexión externa.

En el caso de los Service Test, normalmente se simula el Repository.

Ejemplo:

```
@MockBean
```

```
private CarreraRepository carreraRepository;
```

o en una prueba más aislada:

```
@Mock
```

```
private CarreraRepository carreraRepository;
```

8.2 Qué hace when()

when() permite indicar qué debe responder el mock cuando se invoque un método.

Ejemplo:

```
when(carreraRepository.findById("1"))  
    .thenReturn(Optional.of(carrera));
```

Esto significa:

Cuando alguien llame a findById("1"), devuelve esta carrera simulada.

8.3 Qué hace verify()

verify() permite comprobar que un método fue llamado.

Ejemplo:

```
verify(carreraRepository).deleteById("1");
```

Esto es especialmente útil en métodos que no retornan información, como los métodos de eliminación.

9. Análisis de pruebas en capa Controller

9.1 Qué es la capa Controller

La capa Controller expone los endpoints REST del sistema.

Es la capa que recibe solicitudes HTTP como:

GET
POST
PUT
DELETE

En un proyecto Spring Boot, los controladores suelen usar anotaciones como:

@RestController
@RequestMapping
@GetMapping
@PostMapping
@PutMapping
@DeleteMapping

9.2 Qué se analiza en los Controller Test

En los archivos de Controller Test del proyecto Biblioteca se deben revisar pruebas asociadas a endpoints.

Estas pruebas no buscan validar la base de datos.

Buscan validar:

La ruta

El método HTTP

El código de respuesta

El JSON retornado

La comunicación con el Service simulado

9.3 Uso de MockMvc

MockMvc permite simular peticiones HTTP sin levantar un servidor real.

Ejemplo conceptual:

```
mockMvc.perform(get("/api/estudiantes"))  
    .andExpect(status().isOk());
```

Esto simula una petición:

GET /api/estudiantes

y espera una respuesta:

200 OK

9.4 Uso de jsonPath()

jsonPath() permite validar el contenido del JSON retornado.

Ejemplo:

```
.andExpect(jsonPath("$[0].nombres").value("Juan Pérez"));
```

Esto valida que el primer elemento de la respuesta tenga el nombre esperado.

9.5 Uso de ObjectMapper

ObjectMapper permite convertir objetos Java a JSON.

Se usa principalmente en pruebas POST y PUT.

Ejemplo:

```
.content(objectMapper.writeValueAsString(estudiante))
```

Esto transforma el objeto estudiante en JSON para enviarlo como cuerpo de la petición.

10. Qué se debe observar en cada Controller Test

Para cada prueba de Controller, el estudiante debe responder:

Pregunta	Respuesta esperada
¿Qué Controller se está probando?	Controller correspondiente
¿Qué endpoint se prueba?	Ruta exacta
¿Qué método HTTP se utiliza?	GET, POST, PUT o DELETE
¿Qué Service se simula?	Service usado por el Controller
¿Qué datos se envían?	JSON o parámetros
¿Qué código HTTP se espera?	200, 201, 204, 400, 404, etc.
¿Qué JSON se valida?	Campos esperados
¿Qué podría fallar?	Ruta, método, JSON, código HTTP, mock faltante

11. Posibles errores al ejecutar pruebas

11.1 Error por conexión a base de datos

Puede ocurrir cuando la prueba intenta levantar el contexto completo de Spring y conectarse a MySQL.

Ejemplos de error:

Access denied for user 'root'

Unknown database

Communications link failure

Connection refused

Esto puede ocurrir si el perfil activo usa la base de datos de desarrollo.

11.2 Error por perfil incorrecto

Si el proyecto tiene activo:

```
spring.profiles.active=dev
```

las pruebas podrían intentar usar la configuración de desarrollo.

Para pruebas se recomienda usar:

```
spring.profiles.active=test
```

o ejecutar:

```
mvn test -Dspring.profiles.active=test
```

11.3 Error por ruta incorrecta

Si el Controller tiene:

```
@RequestMapping("/api/v1/clientes")
```

pero el test usa:

```
/api/clientes
```

la prueba fallará con:

```
404 Not Found
```

11.4 Error por código HTTP distinto

Si el test espera:

```
status().isOk()
```

pero el Controller devuelve:

```
201 Created
```

la prueba fallará.

El test debe validar el código real que devuelve el endpoint.

11.5 Error por campo JSON distinto

Si el test valida:

```
jsonPath("$.nombre")
```

pero el JSON retorna:

```
{  
  "nombreCliente": "Juan"  
}
```

la prueba fallará porque el campo esperado no existe.

11.6 Error por falta de mock

En una prueba de Controller con `@WebMvcTest`, si no se simula el Service, Spring no podrá construir correctamente el Controller.

Ejemplo:

No qualifying bean of type ClienteService

12. Relación con DataFaker

La guía de DataFaker muestra cómo generar datos falsos para poblar la base de datos en ambiente de desarrollo.

Esto es útil para:

Probar manualmente desde Swagger

Tener datos iniciales

Evitar ingresar registros uno por uno

Simular escenarios reales

Sin embargo, es importante diferenciar:

DataFaker ayuda a poblar datos en desarrollo.

Mockito ayuda a simular datos en pruebas unitarias.

Una prueba unitaria no debería depender necesariamente de los datos cargados con DataFaker.

13. Transición hacia microservicios

13.1 Qué cambia al pasar a microservicios

En un proyecto monolítico como Biblioteca, todas las capas están dentro de una sola aplicación.

En un proyecto con microservicios, cada servicio tiene su propia responsabilidad.

Ejemplo:

ms-cliente

ms-mascota

ms-servicio

ms-reserva

api-gateway

eureka-server

Cada microservicio debe tener:

Controller

Service

Repository

Model o Entity

DTO

Exception

Config

Tests

13.2 Diferencia principal en testing

En Biblioteca, normalmente se prueba:

Service -> Repository

En microservicios, a veces se prueba:

Service -> Repository

Service -> Cliente externo

Por ejemplo, si un microservicio de mascotas necesita validar que existe un cliente, no debe acceder directamente a ClienteRepository.

Debe comunicarse con el microservicio de clientes mediante un cliente HTTP, como Feign Client o WebClient.

En testing, ese cliente externo también debe ser simulado.

14. Proyecto DoggySpa

El docente está preparando el proyecto:

doggyspa-parent

Este proyecto será utilizado posteriormente como ejemplo para implementar pruebas unitarias en microservicios reales.

Cuando el proyecto esté finalizado, será revisado y se continuará con la implementación de pruebas para cada microservicio.

La idea es que DoggySpa sirva como ejemplo base para que los estudiantes comprendan cómo aplicar testing en una arquitectura de microservicios de manera progresiva.

15. Enfoque esperado para DoggySpa

Cuando se entregue el proyecto DoggySpa, el trabajo de testing se realizará por etapas.

Etapas 1 — Microservicio simple

Se comenzará con un microservicio que no dependa de otros servicios.

Ejemplo:

ms-cliente

Pruebas esperadas:

ClienteServiceTest

ClienteControllerTest

Etapas 2 — Segundo microservicio simple

Luego se trabajará con otro microservicio con lógica propia.

Ejemplo:

ms-servicio

Pruebas esperadas:

ServicioServiceTest

ServicioControllerTest

Etapla 3 — Microservicio con comunicación

Después se trabajará con un microservicio que dependa de otro.

Ejemplo:

ms-mascota

Este microservicio podría necesitar comunicarse con ms-cliente.

Pruebas esperadas:

MascotaServiceTest

MascotaControllerTest

En este caso se deberá simular:

MascotaRepository

ClienteClient

16. Qué se espera que comprendan antes de llegar a DoggySpa

Antes de implementar pruebas en microservicios, el estudiante debe comprender desde Biblioteca:

Qué es una prueba unitaria

Qué es un mock

Qué se prueba en Service

Qué se prueba en Controller

Qué hace MockMvc

Qué valida un assert

Qué valida verify

Qué errores pueden aparecer

Por qué no se debe depender de la base de datos real

Si estos conceptos no están claros, al trabajar con microservicios los errores serán más difíciles de diagnosticar.

17. Actividad sugerida para Biblioteca

Actividad 1 — Análisis de Service Test

Cada equipo debe seleccionar una prueba de Service y completar:

Ítem	Respuesta
Clase de test analizada	
Servicio probado	
Método probado	
Repository simulado	
Datos preparados	
Acción ejecutada	
Assert utilizado	
Resultado esperado	
Posible causa de error	
Cómo se aplicaría en DoggySpa	

Actividad 2 — Análisis de Controller Test

Cada equipo debe seleccionar una prueba de Controller y completar:

Ítem	Respuesta
Clase de test analizada	
Controller probado	
Endpoint probado	
Método HTTP	
Service simulado	
JSON enviado, si corresponde	
Código HTTP esperado	
JSON validado	

Ítem	Respuesta
Posible causa de error	
Cómo se aplicaría en DoggySpa	

18. Actividad de reflexión

Responder en equipo:

1. ¿Por qué no es recomendable probar directamente con la base de datos de desarrollo?
2. ¿Qué ventaja tiene usar mocks?
3. ¿Qué diferencia existe entre probar un Service y probar un Controller?
4. ¿Qué tipo de errores detecta una prueba de Controller?
5. ¿Qué tipo de errores detecta una prueba de Service?
6. ¿Qué cambia cuando un microservicio depende de otro?
7. ¿Por qué en microservicios se debe simular también la comunicación externa?
8. ¿Qué se debería validar antes de afirmar que un endpoint está correctamente probado?

19. Evidencia esperada de la actividad

Cada equipo debe entregar:

1. Análisis de al menos un Service Test.
2. Análisis de al menos un Controller Test.
3. Explicación de los mocks utilizados.
4. Explicación de los asserts utilizados.
5. Identificación de posibles errores.
6. Breve relación con cómo se aplicaría en microservicios.

No se requiere implementar todos los test de Biblioteca desde cero.

20. Criterios de logro sugeridos

Muy buen desempeño

El equipo analiza correctamente las pruebas, identifica la capa probada, explica los mocks, los asserts, los resultados esperados y relaciona claramente el patrón con microservicios.

Desempeño aceptable

El equipo identifica la mayoría de los elementos del test y comprende de forma general cómo se aplicaría en DoggySpa, aunque con algunas explicaciones incompletas.

Desempeño incipiente

El equipo describe parcialmente el test, pero confunde capas, mocks o resultados esperados.

Desempeño no logrado

El equipo no logra explicar qué se prueba, qué se simula ni qué resultado se espera.

21. Cierre de la actividad

Esta primera etapa se enfocará en analizar pruebas ya existentes en Biblioteca.

Posteriormente, cuando el proyecto DoggySpa esté terminado y revisado, se utilizará como base para realizar pruebas unitarias en microservicios reales.

El trabajo futuro consistirá en aplicar el mismo razonamiento, pero ahora considerando que cada microservicio tiene su propia responsabilidad, su propia configuración y, en algunos casos, comunicación con otros microservicios.

El objetivo final es que los estudiantes puedan construir, analizar y defender pruebas unitarias en proyectos Spring Boot con arquitectura de microservicios.